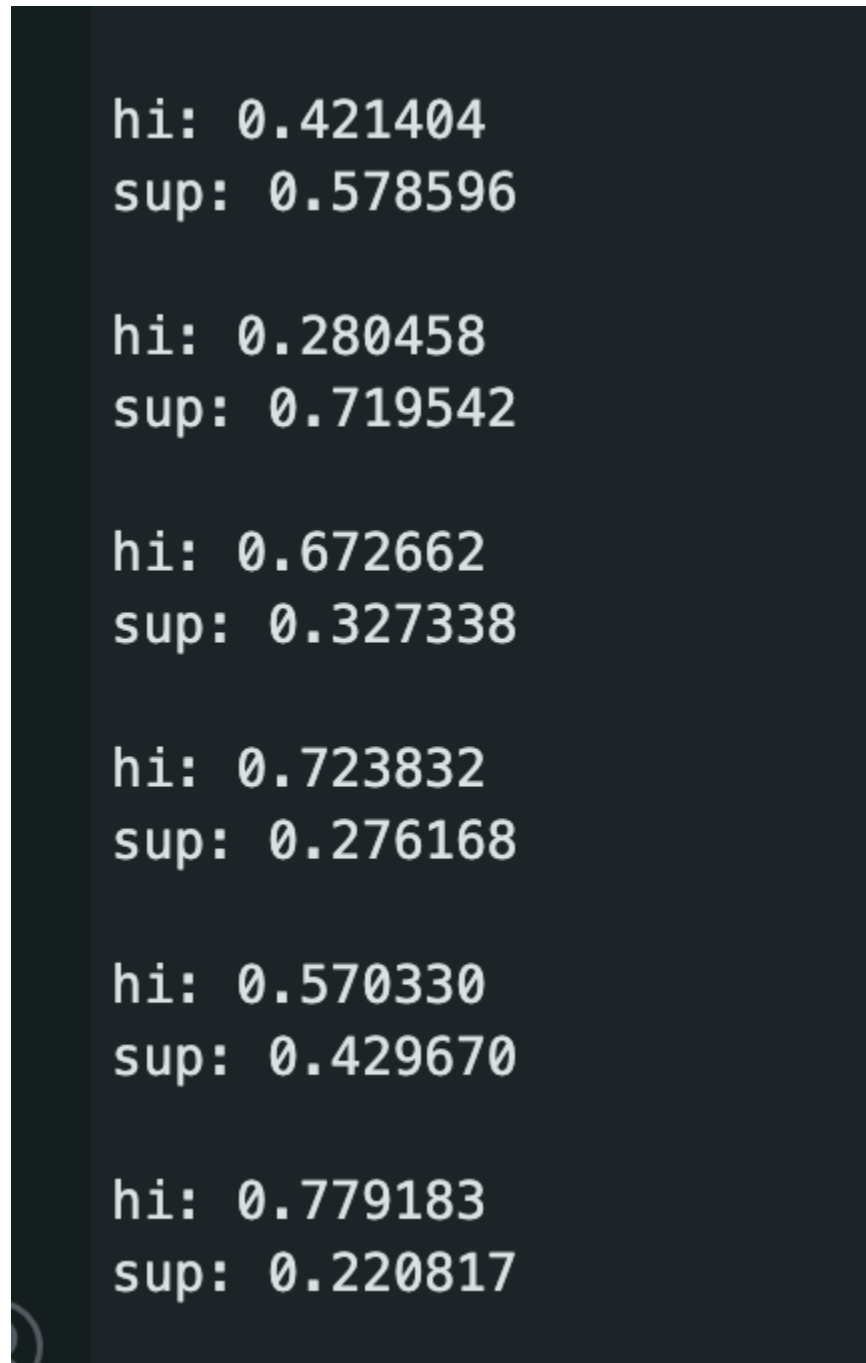


Serial Monitor Screenshot:



Python editing: In the notebook, I modified the compiler line so that the loss function became categorical crossentropy and the metric became accuracy.

python

```
#model.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])  
model.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['accuracy'])
```

I made two adjustments here: swapping the loss function over to categorical cross-entropy and switching the evaluation metric to accuracy. Before this, the model relied on mean squared error (MSE) for its loss and mean absolute error (MAE) as its metric. Those options work well for regression tasks, but they aren't ideal for classification. In this case, the inputs were one-hot encoded and the goal was binary classification, so the model only needed to distinguish between two possible classes. Because MAE is built to gauge prediction error on continuous values, it doesn't really fit as a way to judge classification performance. Accuracy is the better fit since it tells you directly what percentage of samples were classified correctly. In the same way, categorical cross-entropy is purpose-built for classification tasks that use one-hot encoded labels. It captures how closely the predicted probability distribution lines up with the true class distribution, which makes it more effective for training classification models. MSE can technically still be applied to classification, but it's really meant for regression and usually leads to slower learning and weaker classification results than categorical cross-entropy.

The final fix was updating the graph label to accuracy.